1st class

App
↑
OS
↑
Hardware (CPU, memory)
↓
main memory
RAM + ROM

Midterm – 80% theory + 20% calculation/simulation
Final → 30% " + 70 "

27-01-26

* Abstract view the components → Flow of diagram
# what operating systems DO → slide

OS as a Resource Allocator

→ The OS decides how to distribute the limited hardware resource among competing processes

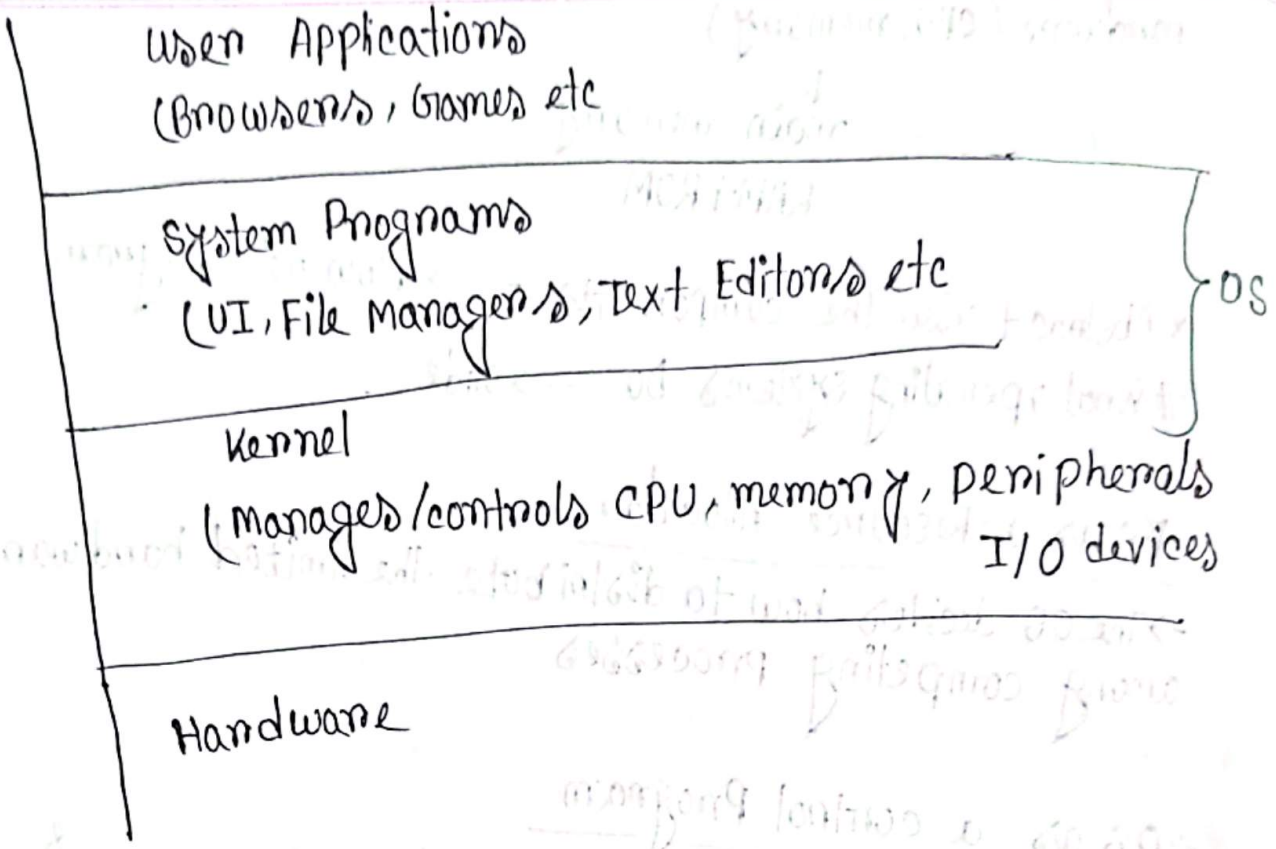
OS as a control Program

The component that controls execution Programs & prevents errors improper use of computer..

Example: Enforce boundaries so one Program cannot overwrite another's memory

Kernel: The core Part of the OS that directly controls hardware resource

OS = Kernel + System Program



Defining operating systems

computer system organization

Shared Bus Architecture

→ The CPU & I/O devices all share the same Path to get to main memory

→ each device has a small buffer where data is stored temporarily

→ The CPU moves data between memory & these buffers

Doing part not need for Slide

→ To keep things efficient the device controller sends an interrupt to the CPU when it's operation is finished

Interrupt → come from hardware (after that Part + DMA start)

example with keyboard

→ When you press a key, the keyboard controller raises and interrupt, which tells the CPU to stop its current work.

→ The CPU then looks into the interrupt vector, which is a table that stores the address of all possible interrupt service routines (ISR) functions

→ Since it is a keyboard, it Jumps to the keyboards ISR, which is a small program in the OS that knows how to handle that device

→ The ISR reads the ~~keyboard~~ keycode from the controller buffer, translates it into a character and gives it to the application Example text editor.

Traps → come from software

→ Not all interrupts come from hardware like keyboards

→ Some are generated ~~by~~ inside the CPU itself

→ These are called Traps / exceptions

→ A Trap is a software generated Interrupt that can occur either because of an error (Example: divided by zero) or because a program makes a request to the OS such as a system call

→ Both hardware interrupts and software traps are handled in the same way
→ Since the OS mainly responds to these events, it is said to be Interrupt driven.

→ As OS is interrupt driven because it does not constantly check devices or programs to see if something happened
→ Instead it mostly sits idle until interrupt occurs.

Interrupt Timeline
" handaling
Storage Structure → next class

class 2nd

1-2-26

RAM: CPU loads instructions only from main memory
→ So any program must first be loaded into main memory to run → limitations
→ Main memory is volatile
→ volatile means that the RAM loses all it, contents when power is turned off.

#storage Device hierarchy → cost / access time / storage capacity
→ SRam based volatile → Primary storage
↳ static

→ Storage Hierarchy → storage devices in computer systems (e.g main memory, hard disk etc) are arranged based on 3 factors i) storage capacity
ii) Access Time
iii) Pricing
→ At the top of the hierarchy are the fastest but smallest & most expensive types of storages like CPU registors & cache → They provide data access in nanos nanoseconds but provide little data.
→ They are also typically volatile.
→ At the bottom of the hierarchy are larger capacity memory devices but with much slower access time.
→ Typically non-volatile
→ The OS & hardware work together to move data between levels (eg caching from disk to RAM) or RAM to disk
↳ Secondary storage ↑↓

so that the CPU can access frequently used data quickly, while still having access to a huge storage capacity as needed.

Page-20

Direct Memory Access (DMA) → start from Interrupt
↳ on programmed I/O

→ In interrupt-driven I/O, the device interrupts the CPU for every byte of data.

→ The CPU must move each piece of data in the I/O buffer into main memory.

→ This work fine for small, slow devices like keyboard, but for large transfers, like playing a movie from the hard disk, the interrupt-per-byte style transfer would severely slow down / choke the CPU.

→ The solution is Direct Memory Access (DMA)

→ Transfers whole block of data directly into RAM & only interrupt the CPU once per block.

OS → check instruction / block → then he went to CPU

- There is no CPU Intervention in-between, and it is free to do other things while data is transfering

Note: Programmed I/O Interrupt happens before data reaches main memory
DMA: After main memory
The CPU acknowledges the transfer to OS

Multiprogramming & Multitasking

Multiprogramming: Multiple Progams are kept in RAM (memory) at the same time.
The OS switches between them when one is waiting (ex: writting NotePad wait for the Prompt)

Goal: keep the CPU busy (no idle time)

Timesharing / multitasking: The CPU switches between the tasks so quickly, it looks like they are all running simultaneously.
Response time < 1 second.

Goal: keep the programs responsive (AT at one time Youtube, Spotify all run at a same time)

memory layout for multiprogrammed system

Dual mode and Multimode Operation

↳ operations To Protect the hardware, the OS needs to distinguish between a task that is executed on behalf of the OS & one that is executed on behalf of the user.

→ A special bit in hardware called mode bit is used to achieve this.

→ Mode bit 0 → kernel mode

→ " " 1 → user mode

→ some instructions that are privileged are only executable in kernel mode

→ Normal user programs run in user mode (1), when they cannot touch critical hardware

→ If a user program wants to do something privileged (like I/O, change memory mapping's) it must make a system call by raising a trap

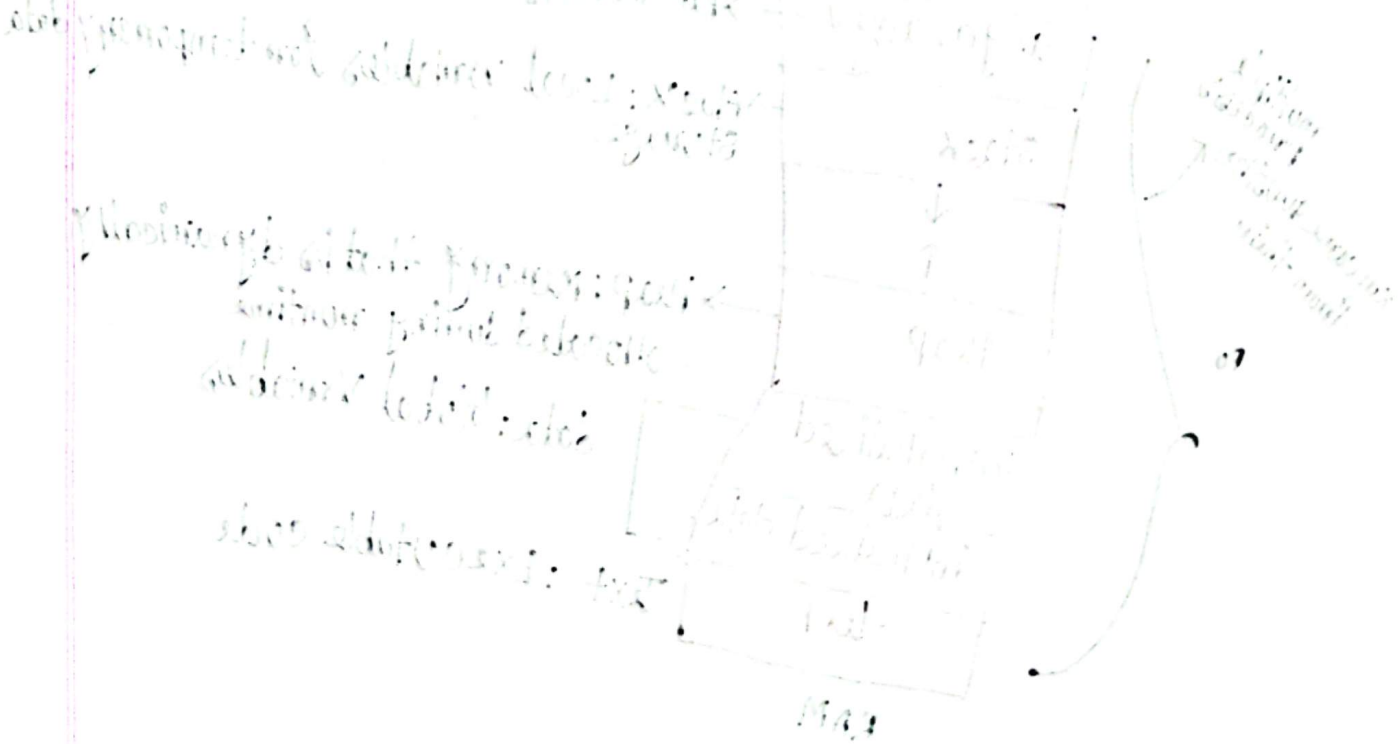
↳ Why trap system call from software

→ When the process executes a trap instruction the CPU switches to kernel mode (0)

→ Before executing kernal code the OS checks things like:

→ Does this Process have Permissions
→ Is this a valid system call number etc.
→ If everything is fine the kernel code executes, mode bit is flipped to user mode (1) and control is returned to user process.

Diagram → Transition from user to kernel mode
→ direct from hardware



class-4

3-02-2

Mode bit 1 but Kernal somehow went con the Program work → no your program runs in user mode. It cannot communicate directly with disk, keyboard, networks etc. (for safety & control)

→ Instead, it makes a System call, which is a controlled door into the kernel

→ operating systems provide an API (Application Programming Interface)

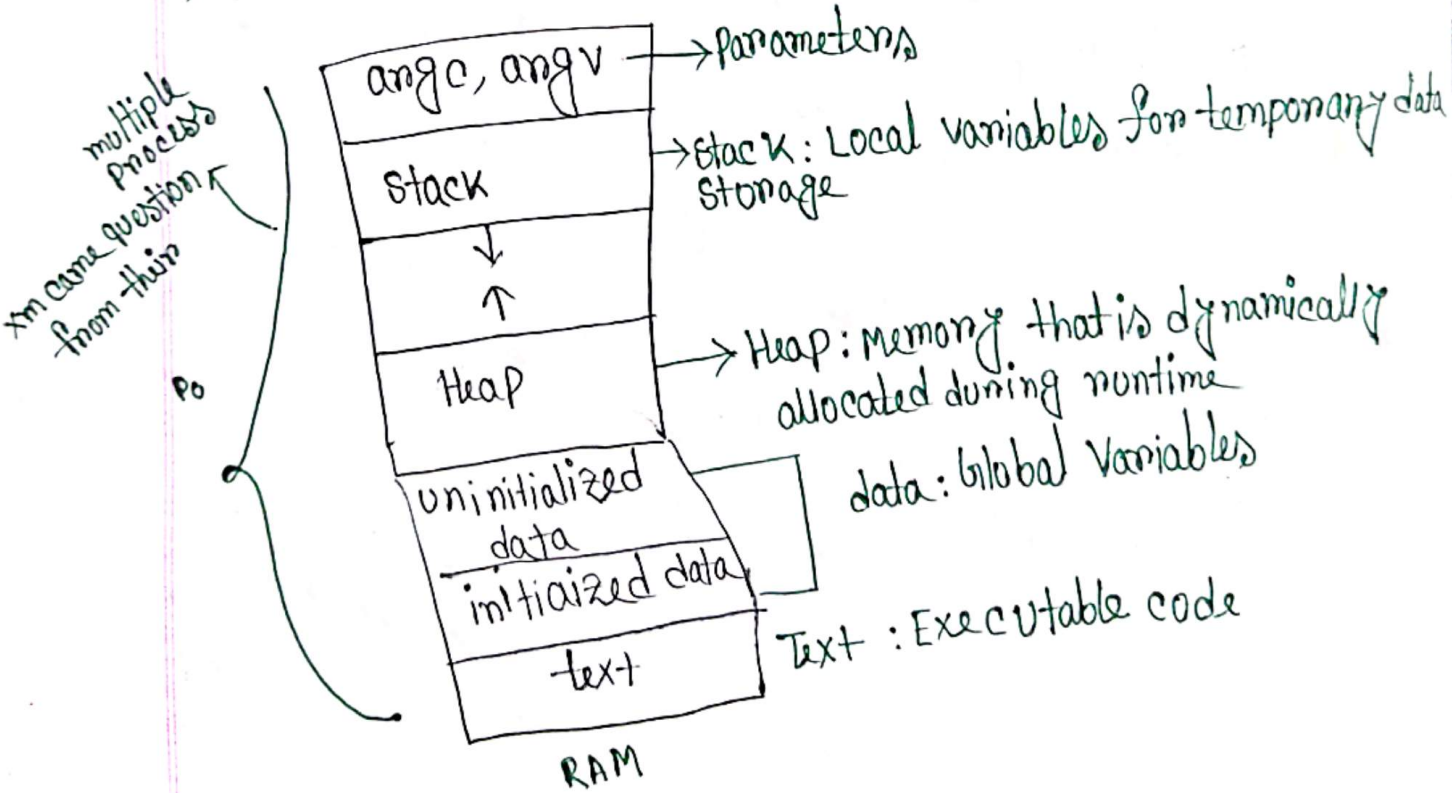
→ a set of library functions in C

Process → Program → Passive → C programming

Active → Chapter-3 : Process → Secondary storage

Process concept

→ A Process is a program in execution (loaded in RAM)



Stack smashing → if hacker rewrite the Previous written that attacke

Process State

Diagram of Process State → queue → FIFO

PCB (Process Control Block (PCB)

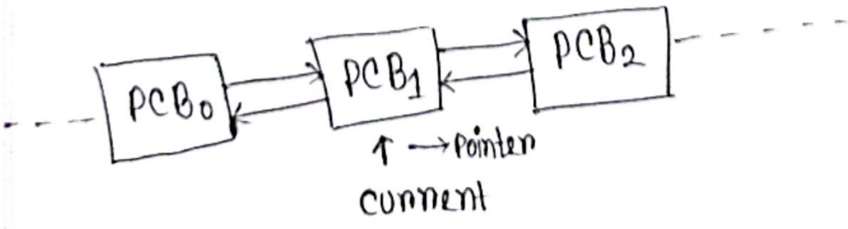
→ Each Process in the system is represented by a process control Block (PCB)

Process State
↓
Process number
↓
Process counter
↓
registers:
↓
memory limits
↓
List of open files

PCB Block → metadata

```
struct PCB{
    Process id;
       "    state;
       "    counter;
       :
}
```

Global Process List:



→ With in the kernel, all active Processes are represented using a doubly linked list of PCB

→ If the kernel wants to manipulate a field in the PCB
e.g change it's state to "terminate"

current → state = "terminate"

→ The number of processes currently in memory is known as the degree of multiprogramming

→ If there are more Processes than cores, excess processes will have to wait untill a core is free.

The Ready Queue & wait queue

Ready queue

→ contains all the processes that are in the ready state
→ Processes here are waiting for CPU allocation
→ On a single core CPU system, there is usually 1 ready queue
- Implemented as a singly linked list.

waiting queue

→contains Process in the waiting (blocked) State, waiting for a specific event

cpu switch-from Process to Process

thread not need now next chapter read

Process scheduling

Schedulers slide

medium term scheduling

c program forking separate Process → code basic (Linux code)

How Processes move between queues

New Process Created: Added to Job Queue & then Ready Queue.

cpu allocated: Process moves from Ready to Running

I/O requested or Time Slice Expired

Process moves from

Running to waiting

or Running to Ready

I/O or event completes: Process moves from waiting to Ready

Process Terminates: Removed from all queues

class-5

10-1-26

Process creation

Process creation:

Parent Process (PID 1000)
code is about to fork()

↓ fork() system call

Kernel creats child Process (PID 1001)
with the copy of parent's memory

After fork() returns:

Parent Process resumes:

Pid = fork() → 1001
Executes parent - specific code

child Process resumes

Pid = fork() → 0
Executes child - specific code

we can use exec() to wipe parents memory with child's specific code.

Process Termination

→ Ideally, we want the Parent to finish first and call wait() for child's exit status.

→ But if the child finishes first because the Parent was still running/busy/blocked, the child's exit status won't be picked up by the Parent.

→ The child will still occupy the global Process list as a PCB, taking up (small) memory.

→ The process is said to have turned into a zombie

→ However, it is no longer in the user space. When a process calls exit(), all it's code, data, stack, heap are released.

→ If the Parent crashes/terminates without invoking wait(). the process becomes an orphan

→ Init process adopts the orphan & calls wait on the Parent's behalf.

Inter Process communication (IPC)

→ A Process is independent if it doesn't share data with other processes executing in the system.

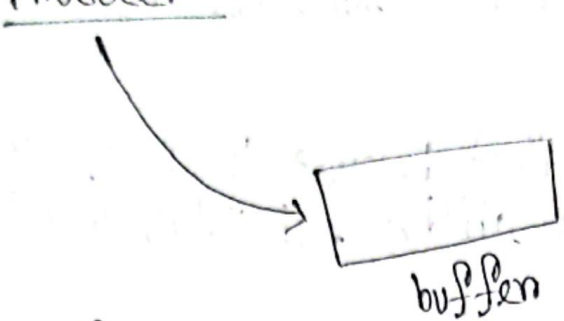
→ Any process that show data with other Process is a cooperating process.

2 ways to share data (for cooperating Processes)

① Shared memory: Harder to implement. Faster Doesn't required system calls.

② Message passing: Easier to implement: Slower. Requires system calls.

Producer consumer Architecture in Shared Memory

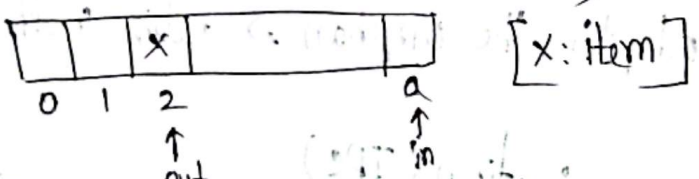


If buffer is bounded, the Producer needs to wait if buffer is full
" consumer " " " " " " empty

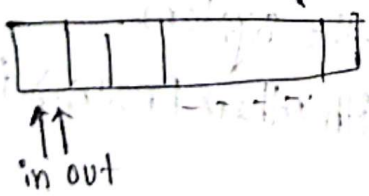
→ The shared buffer is implemented as a circular array with 2 pointers in & out.

in → Points to the next free location for the Producer to Produce an item

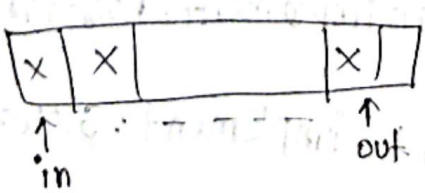
out - Points to the first full Position in the buffer to Process an item (for the consumer)



→ The buffer is empty when in == out



→ The buffer is full when (in + 1) % Buffer-Size == out



When a Producer Puts an item:

```
buffer[in] = next_Produced;
in = (in + 1) % BUFFER_SIZE;
```

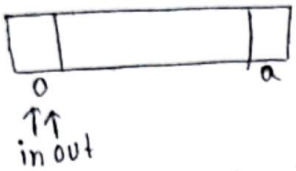
When consumers an item:

```
next_consumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;
```

Simulation Example

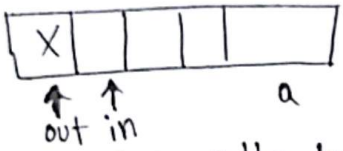
→ Initially, buffer is empty

in = 0, out = 0

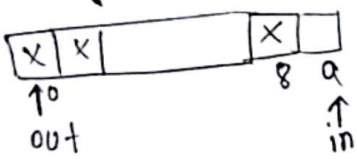


→ After 1 item is added

in = 1, out = 0



→ After filling up the buffer with 9 out of 10 Items in = 9, out = 0



→ If the Producer added 1 more item, in would update to 0 which would make in == out we have already used this condition to mean empty

∴ Solution: leave one slot empty to denote Full

15-02-26

chapter-2 (Threads)

Threads and concurrency

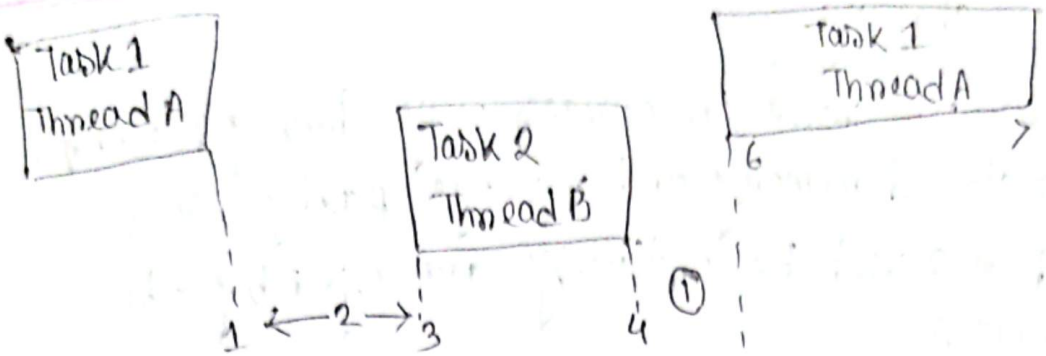
A thread is a basic unit of cpu utilization. It consists of Thread ID, Program counter (PC), register set and a stack

→ it shares with other threads belonging to the same process its code section, data section and other OS resources like files.

→ A traditional Process is single-threaded ex it has a single thread of control

- If a process is multithreaded, it can perform multiple tasks concurrently (using a single core CPU) or in Parallel (using multicore/multiProcessor) CPUs.
- on a single-core CPU multithreading may not always make a program complete faster in fact it can make it slower.
- When you run a multithreaded Process on a single-core CPU, the core cannot excute 2-threads simultaneously

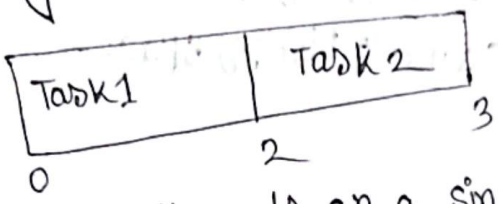
→ The scheduler rapidly switches between threads each gets a short time slice.



context switch overhead

Total CPU time = Time (Thread A) + context + Time (Thread B) + switch overhead

If you used single thread



*So why even use threads on a single core?

→ Improves system responsiveness, resource sharing, economy, scalability.

Better responsiveness

example: A text editor

Thread 1: Handles keyboard input

Thread 2: Saves files in the background

App feels responsive because while Thread 2 wait for disk I/O, the scheduler gives cpu time to Thread 1.

Resource sharing

→ Process can share resources only through techniques such as shared memory and message passing such techniques must be explicitely arranged by the Programmer.

→ Threads share the memory and the resources of the Process to which it belongs to by default.

→ The benefit of sharing code and data is that, it allows an application to have several different threads of activity within the same address space.

Economy

- Thread creation consumes less time and memory than Process creation.
- context switching is faster between threads than Processes.

Scalability

- In multicore or multiprocessor systems, threads can run in parallel on different processing cores.

Speed up = $\frac{1}{(1-P) + P/N}$

(1-P): sequential, P/N: Parallel

$= \frac{1}{S + \frac{(1-S)}{N}}$

int a = 1 + 2i
int b = 3 * 3i } can be Parallel
int c = a + bi

1/3 → series/sequential

2/3 → Parallel

if 1 core $= \frac{1}{1/3 + \frac{1-1/3}{1}}$

$= 1$

if 2 core $= \frac{1}{1/3 + \frac{1-1/3}{2}}$

$= 1.5$

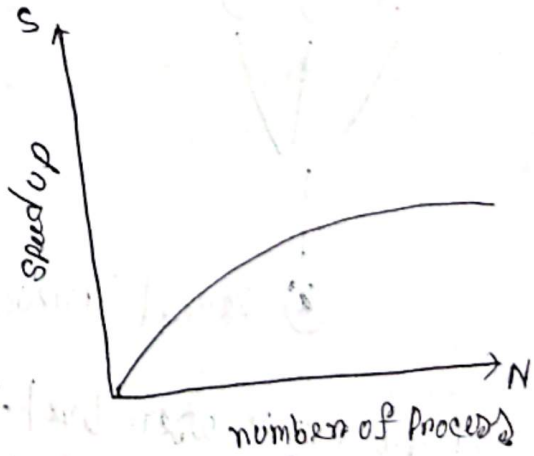
#In reality, N cannot be more than 2 for these example

N = 10000

Speed up $= \frac{1}{1/3 + \frac{1-1/3}{1000}}$

$= 2.999$

Speedup range $\leq 1/3$



speedup max $= \frac{1}{1-P}$

Speed up = $\frac{1}{(1-P) + P/N}$

sequential, Parallel

$= \frac{1}{S + \frac{(1-S)}{N}}$

int a = 1 + 2i
int b = 3 * 3i } can be Parallel
int c = a + bi

1/3 → series/sequential

2/3 → Parallel

if 1 core $= \frac{1}{1/3 + \frac{1 - 1/3}{1}}$

$= 1$

if 2 core $= \frac{1}{1/3 + \frac{1 - 1/3}{2}}$

$= 1.5$

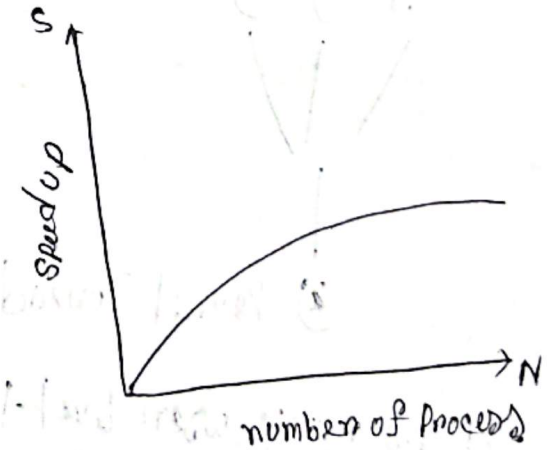
#In reality, N cannot be more than 2 for these example

N = 10000

Speed up $= \frac{1}{1/3 + \frac{1 - 1/3}{1000}}$

$= 2.999$

Speedup range $\leq 1/3$



speedup max $= \frac{1}{1-P}$

Types of Threads

① User threads – managed without kernel support

② Kernel Threads – Managed directly by the OS

There must exists a relationship between user-threads & kernel threads.

3 ways to establish this relationship

① Many-to-one Model
② one to one model
③ Many to many model

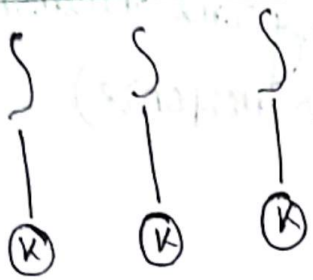
Many to one model

→ Maps many user level threads to one kernal-thread
→ Thread management such as context switch is done in the user space, so it is fast.

→ The entire Process will block if a thread makes a blocking system call

→ Because only one thread can access in kernel at a time, multiple threads are unable to run in parallel on multiprocessors.

one to one Model



- maps each user thread to a kernel thread
- Allows another threads to run while one is making a blocking system call.
- Also allows multiple threads to run in Parallel on multiprocessors
- creating a user thread requires creation of the corresponding kernel thread, which may burden the performance of the system.

Example : User Threads: T_1, T_2, T_3, T_4, T_5

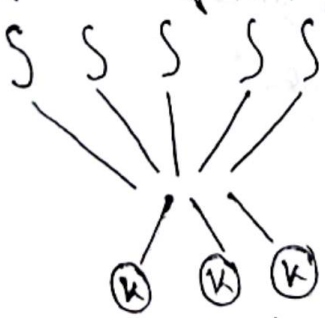
Kernel Threads: K_1, K_2

2 CPU cores

core 1 | T_1 | T_2 | T_1 | T_2 |

core 2 | T_3 | T_4 | T_5 | T_3 | T_4 | T_5 |

Many to Many model

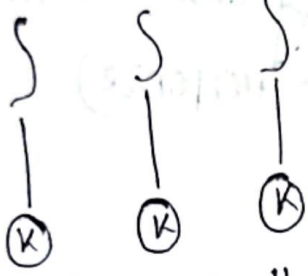


→ Outcomes the limitations of the Previous 2 models

→ The entire Process will block if a thread makes a blocking system call

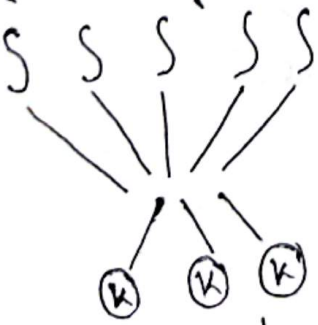
→ Because only one thread can access in kernel at a time, multiple threads are unable to run in parallel on multiprocessors.

one to one Model



- maps each user thread to a kernel thread
- Allows another threads to run while one is making a blocking system call.
- Also allows multiple threads to run in Parallel on multiprocessors
- creating a user thread requires creating the corresponding kernel thread, which may burden the performance of the system.

Many to Many model



→ Outcomes the limitations of the Previous 2 models

Example: User Threads: T_1, T_2, T_3, T_4, T_5

Kernel Threads: K_1, K_2

2 CPU cores

core 1 | T_1 | T_2 | T_1 | T_2 |

core 2 | T_3 | T_4 | T_5 | T_3 | T_4 | T_5 |